

Taller 2 – Circuitos Combinacionales en VHDL

1. Objetivo

El Taller 2 consistirá en desarrollar todos los circuitos combinatorios necesarios para realizar una ALU de N bits.

El desarrollo se hará en *VHDL* por medio de la plataforma online *EDA Playground*.

2. Enunciado

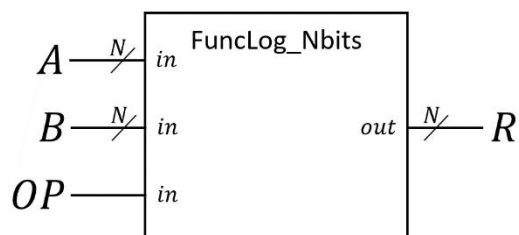
Requisitos generales:

- ✦ Para cada apartado desarrolle un proyecto distinto.
- ✦ Respete los nombres y las características de cada entrada o salida definida.
- ✦ Utilice descripciones a nivel RT en todos los casos que pueda y descripciones a nivel compuertas para el diseño del módulo lógico.
- ✦ Cada diseño debe tener su respectivo *testbench*. Para la ALU utilice la tabla de referencia adjunta. Para los otros dos módulos proponga los casos que considere necesario. Para todos los testbench tome $N = 4$.
- ✦ En todos los diseños utilice *constant* y *generic* para mantener el código legible y generalizado. Para la simulación de los casos (testbench) utilice la sentencia *assert*.

Complete cada uno de los códigos solicitados en los siguientes apartados:

A. Módulo Lógico (*FuncLog_Nbits*)

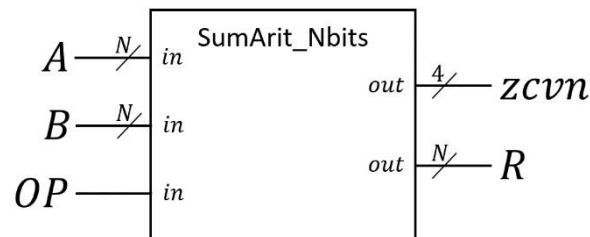
Tiene 3 entradas, **A** y **B** de N bits tipo *signed* y **OP** de 1 bit tipo *std_logic*. Su única salida **R**, de N bits tipo *signed*, será el resultado de hacer una operación AND bit a bit cuando **OP** sea 0 o una operación **OR** bit a bit cuando **OP** sea igual a 1.



B. Módulo Aritmético (*SumArit_Nbits*)

Tiene 3 entradas, **A** y **B** de **N** bits tipo *signed* y **OP** de 1 bit tipo *std_logic*. Su salida **R**, de **N** bits tipo *signed*, será la suma de **A** y **B** cuando **OP** sea 0 y la resta cuando **OP** sea igual a 1. La salida **zcvn**, de 4 bits tipo *std_logic_vector*, deberá responder a los *flags* *cero*, *carry*, *overflow* y *negativo*:

- ✦ El resultado es cero $\Leftrightarrow$ **zcvn(3)** vale 1
- ✦ La suma binaria produjo acarreo (o borrow en la resta) $\Leftrightarrow$ **zcvn(2)** vale 1
- ✦ La suma (o resta) interpretada en complemento a 2 dio overflow $\Leftrightarrow$ **zcvn(1)** vale 1
- ✦ El resultado interpretado en complemento a 2 es negativo $\Leftrightarrow$ **zcvn(0)** vale 1

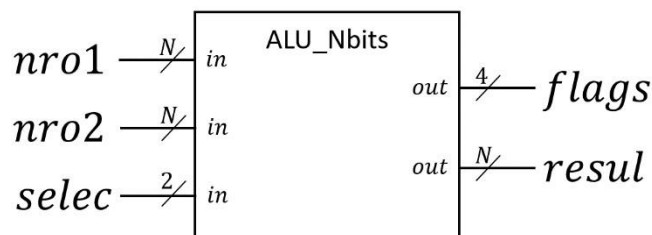


C. ALU (*ALU_Nbits*)

Tiene 3 entradas, **nro1** y **nro2** de **N** bits tipo *signed* y **selec** de 2 bits tipo *std_logic_vector*. La salida **resul**, de **N** bits tipo *signed*, responde a la siguiente lógica:

- ✦ Si **selec** = 00: $resul = A + B$
- ✦ Si **selec** = 01: $resul = A - B$
- ✦ Si **selec** = 10: $resul = A \text{ AND } B$
- ✦ Si **selec** = 11: $resul = A \text{ OR } B$

Además, tiene una salida de *flags* de 4 bits tipo *std_logic_vector* con igual funcionalidad que la salida **zcvn** del módulo aritmético.



***Para el caso de la ALU debe instanciar los módulos lógicos y aritméticos desarrollados previamente, para luego interconectarlos.**

3. Verificación Funcional

Complete la siguiente tabla con los casos de prueba obtenidos del *testbench* de la ALU para garantizar el correcto funcionamiento de su diseño.

Operandos		Operación	Salidas				
A	B	OP	S	Z	C	V	N
1111	0001	00					
0101	0101	00					
0101	0101	01					
1000	0111	00					
1000	0111	01					
0110	1010	00					
0110	1010	01					
1101	1100	10					
1101	1101	11					

4. Entrega Final del Taller

La entrega del taller consta de un pequeño informe y los todos los códigos realizados en EDA Playground (enlace web).

- ✦ Todos los códigos deberán tener el número de grupo y comisión en cada uno de los mismos y figurar en el informe.
- ✦ Para el informe se debe completar el archivo UNRN-ED_T2_ComX_GrupoXX.docx Este archivo deberá ser subido al Campus Bimodal, en pdf con el correcto número de grupo y comisión, que ya le ha sido asignado.